EMBEDDED SYSTEMS / ARM CORTEX-M3

MYRTOS

Real-Time Operating System Kernel Design, Scheduling Architecture & ARM Cortex-M3 Porting

Engineering Design & Implementation Report | Source-code-based draft | 12 October 2026

Prepared by Mohamed Al Dreamly

DOCUMENT STATUS: Draft for technical review — not a claim of completed system validation.

01 Executive Summary

MYRTOS is a custom educational real-time operating system kernel implemented in C and ARM inline assembly for Cortex-M3. The supplied code covers task descriptors, dedicated process stacks, a priority-oriented scheduler with a FIFO ready queue, exception-driven scheduling paths, a SysTick-based delay mechanism, and a basic single-waiter mutex. This report documents the implemented architecture and intended behavior while distinguishing source-observed behavior from unverified runtime outcomes.

The strongest low-level work is the explicit treatment of MSP/PSP, CONTROL, SVC exception decoding, PendSV-based context switching, and synthetic initial task frames. Several correctness and robustness issues remain for review before describing the kernel as production-ready.

02 Scope, Platform & Source Baseline

Area	Source baseline / configuration
Core	ARM Cortex-M3; core_cm3.h CMSIS dependency
Kernel	Scheduler.c / Scheduler.h
Architecture port	CortexMX_OS_porting.c / .h
Queue interface	MYRTOS_FIFO.h; FIFO implementation not included in this review
Clock / tick	Code assumes 8 MHz CPU clock and calls SysTick_Config(8000), nominal 1 ms
Task capacity	OSTasks[100] and Ready_QUEUE_FIFO[100]; no checked task-count guard in CreateTask

Source note: Two uploaded Scheduler.c copies were supplied. The report describes their visible implementation. Keil runtime traces, linker map, target startup code and test mains are outside the current evidence set.

03 Kernel Architecture

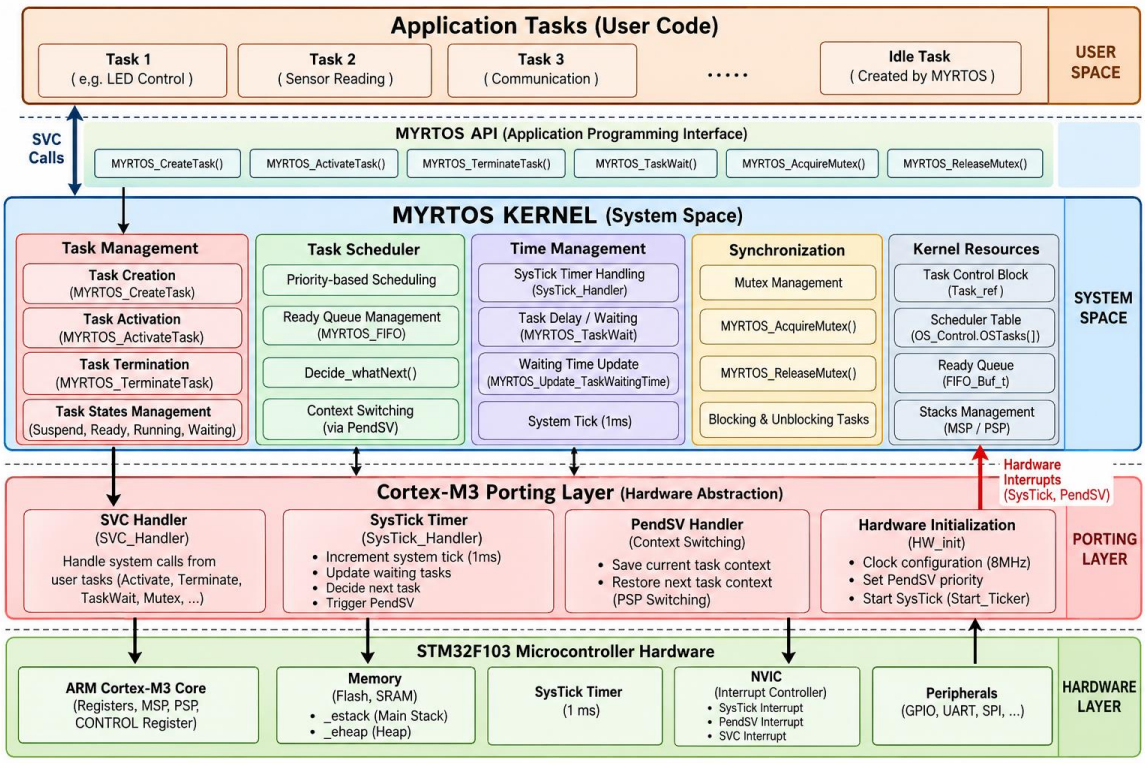


Figure 1. MYRTOS layered architecture, adapted to implemented modules.

The kernel is organized around a global OS_Control object, an OSTasks registry, a FIFO-backed ready queue, an idle task, and Cortex-M exception handlers. Public task APIs change task states and invoke SVC; SysTick updates delay counters and selects the next task; PendSV is intended to save and restore execution context.

Component	Responsibility	Primary symbols
Kernel control	Mode, stack locator, current/next task	OS_Control, MYRTOS_init
Task management	Allocate stacks, initialize context, register and activate tasks	MYRTOS_CreateTask, MYRTOS_ActivateTask
Scheduler	Priority sort, queue rebuild, select next task	bubbleSort, MYRETOS_Update_Schaduler_table, Decide_whatNext
Architecture port	Exception dispatch, tick, context switch, privilege/stack selection	SVC_Handler, PendSV_Handler, SysTick_Handler
Services	Timed blocking and basic mutex ownership handoff	MYRTOS_TaskWait, MYRTOS_AcquireMutex / ReleaseMutex

Conceptual control flow

Application API → task state update → SVC dispatch → scheduler-table update / ready queue → NextTask selection → pending PendSV → save current PSP context → restore next PSP context → exception return. Independently: SysTick → delay update → selection → PendSV request.

04 Task Control Block & Lifecycle

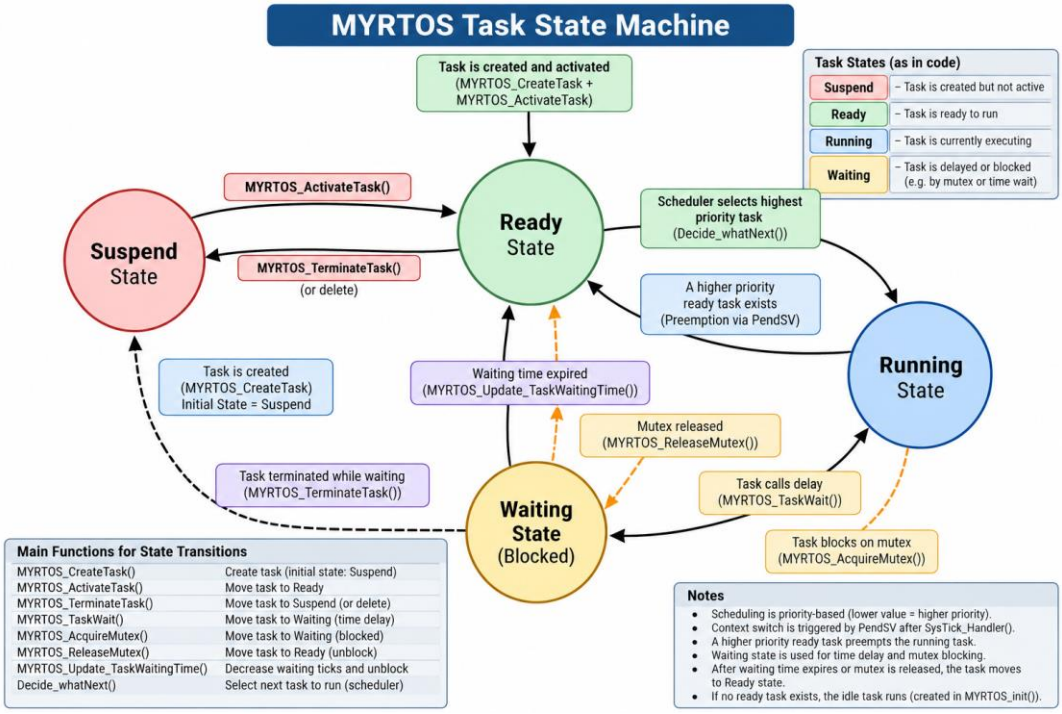


Figure 2. Task-state transitions reflecting source naming. Suspend also represents blocked tasks; diagram is conceptual.

Task_ref stores stack size, priority, entry function pointer, AutoStart field, PSP bounds, saved PSP pointer, task name, state, and a tick-delay structure. Defined states are Suspend, Running, Waiting, and ready. The code uses Waiting as a runnable/activated state and Suspend for both terminated and blocked tasks; this dual use should be explained explicitly or refactored to avoid semantic ambiguity.

MYRTOS_CreateTask reserves a descending stack range from PSP_Task_Locator, checks its lower bound against _eheap, synthesizes an initial stack frame, adds the descriptor to OSTasks, and initializes the task to Suspend. AutoStart is present in the descriptor but is not acted on in the supplied implementation.

05 Memory & Stack Design

MYRTOS_Create_MainStack reserves MainStackSize = 3072 bytes below linker symbol _estack for MSP and sets the initial PSP allocation locator below it. Task stacks are then placed in descending RAM addresses with an eight-byte gap between reservations. The linker heap symbol _eheap is used as a lower-bound check; the code does not fully validate alignment or protect against exceeding the OSTasks array.

Initial task frame

MyRTOS_Create_TaskStack writes xPSR = 0x01000000 (Thumb bit), PC = task entry, LR = Task_Exit_Error, followed by 13 zero words, yielding 16 words (64 bytes) in the initial context. This is intended to combine the eight-word hardware exception frame and eight software-saved registers R4–R11. The initial stack pointer must satisfy the ARM exception-return alignment contract; current arithmetic does not guarantee that for arbitrary Stack_Size values.

On exception entry (hardware frame)	Kernel-managed context
R0, R1, R2, R3, R12, LR, PC, xPSR	R4–R11 (software save/restore)
Pushed automatically on exception entry	Stored on each task PSP by PendSV code

06 Scheduling Policy & Ready Queue

The task registry is sorted using bubbleSort in ascending numeric priority, so lower numbers mean higher scheduling priority. MYRETOS_Update_Schaduler_table drains the FIFO and tries to populate the highest-priority runnable cohort. Decide_whatNext dequeues the next task and re-enqueues the prior task when priorities match, implementing the intended round-robin rotation.

Important distinction: round-robin fairness is the design intent, not yet a proven invariant. The current queue rebuild can terminate prematurely when an adjacent task is suspended; it also accesses PnextTask even when NULL at the end of the task registry. Empty-queue and CurrentTask/NextTask handling also require guards. These cases can affect task eligibility and scheduling correctness.

07 Cortex-M3 Porting & Exception Model

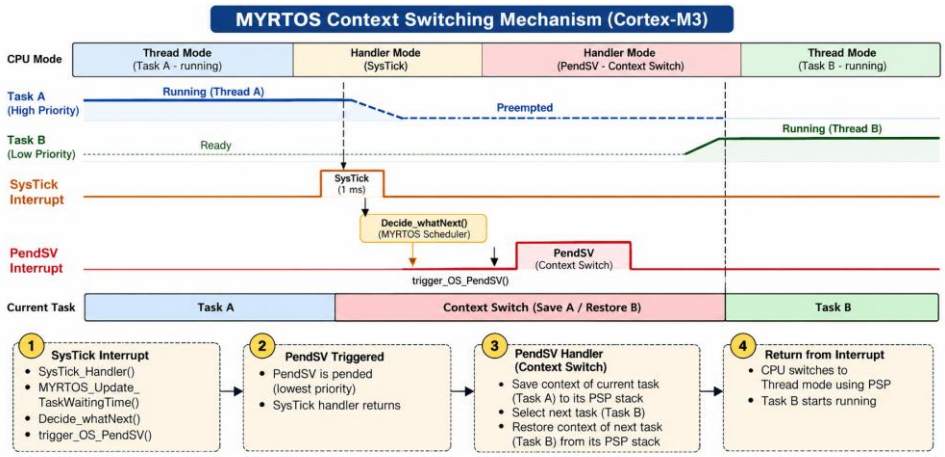


Figure 3. Cortex-M3 exception and context-switching pathways (conceptual; correctness pending validation).

MSP, PSP and CONTROL

The port exposes inline assembly macros to read/write PSP and change CONTROL.SPSEL / CONTROL.nPRIV. The start path selects PSP for Thread mode and attempts unprivileged execution. Handler mode continues using MSP. A privileged exception path is required to regain privilege from unprivileged Thread mode; directly clearing CONTROL.nPRIV from unprivileged Thread mode is not sufficient. Instruction synchronization after CONTROL changes should be reviewed.

SVC — system-call entry

The naked SVC_Handler inspects EXC_RETURN bit 2 to select MSP or PSP and branches to OS_SVC. OS_SVC reads the SVC immediate at stacked PC minus two bytes and dispatches activate, terminate, and waiting-time cases. Mutex SVC identifiers are defined but are not implemented in the OS_SVC switch.

SysTick — tick-driven scheduling

Start_Ticker calls SysTick_Config(8000); at an actual 8 MHz SysTick clock this corresponds to a 1 ms interval. SysTick_Handler updates waiting times, invokes Decide_whatNext, and pends PendSV. The actual clock tree and exception priority configuration must be verified against the target startup and CMSIS implementation; comments alone do not establish SysTick priority.

PendSV — context switching

PendSV_Handler reads PSP, intends to preserve R4–R11 for the current task, updates CurrentTask from NextTask, restores the saved registers, writes PSP, and returns via BX LR. The supplied function is declared naked but mixes ordinary C statements and inline assembly; this is unsafe under compiler register allocation and should be rewritten as a pure assembly wrapper or a carefully separated assembly/C implementation before reliability claims.

08 RTOS Services

Non-busy-wait task delay

MYRTOS_TaskWait sets a task's Blocking flag, stores NoTICKS, marks it Suspend and requests rescheduling via SVC. SysTick decrements blocked tasks' counters and marks a task Waiting when its count reaches zero. This is a tick-based blocking design rather than a polling delay. The NoTICKS = 0 case underflows the unsigned counter, and invoking SVC from SysTick is a control-flow concern to review.

Basic mutex

MYRTOS_AcquireMutex assigns the owner if free; otherwise it records one waiting task, suspends it and requests scheduling. MYRTOS_ReleaseMutex attempts to transfer ownership to that waiter and reactivate it. Current implementation permits only one waiter, has no priority inheritance or ownership verification, and dereferences a NULL owner after release if no waiter exists. Therefore it is a prototype synchronization service, not a fully validated mutex.

Idle task

The idle task is assigned priority 255, toggles a diagnostic variable, and executes WFE in a loop. This provides a low-priority fallback in the intended scheduler design; event behavior and wakeup sequencing should be confirmed during runtime testing.

09 Initialization & Runtime Sequence

- MYRTOS_init: enter OSsuspend mode, reserve MSP region, initialize FIFO, create idle task.
- Application: create and activate tasks through the exposed task APIs.
- MYRTOS_STARTOS: set OSRunning, select idle as current, activate idle, start SysTick, set PSP, select PSP/unprivileged Thread mode, enter idle entry.
- SVC and SysTick: update scheduling state; PendSV is pended for intended context switches.
- PendSV: preserve previous context, switch task pointer and PSP, restore the next task context.

Startup caveat: the initial transition into the first task is performed by a direct idle function call rather than a dedicated exception-return launch path. This merits detailed stack-frame and execution-mode review.

10 Engineering Review — Known Risks

Priority	Observation / required action
Critical	PendSV naked function contains C and inline assembly; validate generated machine code and use assembly-safe context switch.
Critical	Scheduler accesses PnextTask when NULL; add bounds checks and skip suspended tasks safely.
Critical	Mutex release dereferences NULL when no waiter; enforce owner and waiter checks.
High	Decide_whatNext assumes non-NULL current/next task and successful dequeue.
High	SVC called from SysTick during delay wakeup; restructure ISR-safe reschedule signaling.
High	Ensure 8-byte stack alignment, task capacity checks and sufficient per-task frame space.
Medium	Handle zero-tick delay, queue error propagation, and idle/startup transitions.
Medium	Validate clock assumptions, SysTick/PendSV priorities and CONTROL barriers.
Medium	AutoStart declared but unused; clarify state naming and mutex limitations.

11 Validation Boundary & Next Report

This document is a design and code-inspection report, not a test certificate. It does not claim successful register preservation, scheduling fairness, priority preemption, bounded response time, or fault-free operation. The separate MYRTOS Functional Testing & Validation Report will map each Keil test case to setup, source version, expected sequence, debugger evidence, observed outcome, and pass/fail assessment.

12 Source Traceability

Source	Sections supported
Scheduler.c	Task management, stacks, scheduler, context switch, SVC dispatch, delay, mutex, startup
Scheduler.h	Task_ref, Mutex_ref, state and API definitions
CortexMX_OS_porting.c	SVC entry, SysTick, PendSV trigger, clock assumptions
CortexMX_OS_porting.h	PSP/MSP, CONTROL macros, linker symbols
MYRTOS_FIFO.h	FIFO task-pointer interface and status types

Prepared for review and approval. Any technical changes after this draft should be recorded against the corresponding source revision.